

0.1 Finite Volume Method

The finite volume method (FVM) is a widely used discretization method for various problems governed by conservation laws. The FVM is particularly robust and well-suited for arbitrary complex geometry. A key advantage of FVM that benefits this simulation is its natural enforcement of local mass conservation.

To begin with, we consider a general hyperbolic conservation law regarding a scalar variable u as

$$\begin{cases} \frac{\partial u(\mathbf{x}, t)}{\partial t} + \nabla \cdot \mathbf{f}(u; \mathbf{x}, t) = 0, & t > 0, \mathbf{x} \in \Omega, \\ u(\mathbf{x}, 0) = u_0(\mathbf{x}), & \mathbf{x} \in \Omega, \end{cases} \quad (1)$$

where we assume the domain $\Omega \subset \mathbb{R}^2$. The term $\mathbf{f}$ represents the flux transporting the scalar variable u . The basic semi-discretized formulation of the finite volume method can be written as

$$\frac{\partial \bar{u}}{\partial t} + \frac{1}{|E|} \int_{\partial E} \mathbf{F}(\mathbf{x}, t) \cdot \boldsymbol{\nu} d\mathbf{x} = 0, \quad (2)$$

where the cell-averaged value $\bar{u}$ is calculated as

$$\bar{u} = \frac{1}{|E|} \int_E u(\mathbf{x}, t) d\mathbf{x}. \quad (3)$$

Here, $\mathbf{F}(\mathbf{x}, t)$ denotes the approximated numerical flux. We employ the Lax-Friedrichs numerical flux

$$\mathbf{F}(u^+, u^-) = \frac{1}{2}[(\mathbf{f}(u^+) + \mathbf{f}(u^-)) \cdot \boldsymbol{\nu} - \alpha_{LF}(u^+ - u^-)], \quad (4)$$

where u^+ and u^- denote the values approximated from the perspective of element $E^+ \in \mathcal{T}_h$ and $E^- \in \mathcal{T}_h$ respectively, The unit normal vector $\boldsymbol{\nu}$ points from E^+ to E^- , and is orthogonal to the edge. The parameter α_{LF} is the Lax-Friedrichs stabilizer factor. Depending on the available information about the solution, there are two possible approaches for computing α_{LF} :

- The global Lax-Friedrichs Stabilizer: $\alpha_{LF} = \max_{\mathbf{x} \in \Omega} |\partial(\mathbf{f}(\mathbf{x}) \cdot \boldsymbol{\nu}) / \partial u|$;
- The local Lax-Friedrichs Stabilizer: $\alpha_{LF} = \max(|\partial(\mathbf{f}^+ \cdot \boldsymbol{\nu}) / \partial u^+|, |\partial(\mathbf{f}^- \cdot \boldsymbol{\nu}) / \partial u^-|)$.

The numerical flux $\mathbf{F}$, as defined in equation (4), is consistent and conservative across element interfaces. We now replace the u^+ and u^- with reconstructed values $\mathcal{R}^+$ and $\mathcal{R}^-$ and express the numerical flux as

$$\mathbf{F}(\mathcal{R}^+, \mathcal{R}^-) = \frac{1}{2}[(\mathbf{f}(\mathcal{R}^+) + \mathbf{f}(\mathcal{R}^-)) \cdot \boldsymbol{\nu} - \alpha_{LF}(\mathcal{R}^+ - \mathcal{R}^-)]. \quad (5)$$

Here, the reconstructed value $\mathcal{R}$ is of higher-order accuracy. We evaluate the integrals over each edge $e_i \in \partial E$ with a suitable quadrature rule. Let $|e_i|$ denote the length of edge e_i and $|E|$ denote the area of the cell E with Assuming a quadrature rule with G points per edge, the semi-discretized formulation becomes:

$$\bar{u}_t + \sum_{i=0}^4 \frac{|e_i|}{|E|} \sum_{g=0}^G \omega_{i,g} \mathbf{F}(\mathcal{R}_{i,g}^+, \mathcal{R}_{i,g}^-) \cdot \boldsymbol{\nu}_i = 0, \quad (6)$$

where $\omega_{i,g}$ represents the weight associated to the g^{th} quadrature points on edge e_i .

0.2 The Multi-level Weighted Essentially Non-Oscillatory (ML-WENO) Method

We develop an accurate, versatile, robust and efficient finite volume multi level weighted essentially non oscillatory (ML-WENO) reconstruction weighting scheme in Arbogast et al. (2024). The ML-WENO method is primarily used to reconstruct values on both sides of the element interfaces.. It can also be applied to a general reconstruction of discontinuous variables, e.g., piece-wise constant discontinuous Darcy pressure arising due to zero and nonzero porosity regions.

0.2.1 Stencil Polynomials

Given a rectangular stencil $\mathcal{S} = \{E_{i,j}, i \leq s, j \leq t\} \subset \mathcal{T}_h$ on a quasi uniform logically rectangular mesh, which covers the area $\Omega_{\mathcal{S}} = \cup_{E \in \mathcal{S}} \Omega_E$. For each stencil, we construct a tensor product stencil polynomial $Q_{s,t}^{\mathcal{S}} \in \mathbb{Q}_{s-1,t-1}(\Omega_{\mathcal{S}})$. The scaled tensor product stencil polynomial is expressed as

$$Q_{s,t}^{\mathcal{S}}(x, y) = \sum_{p < s, q < t} c_{p,q} \left(\frac{x - x_{\mathcal{S}}}{h_{\mathcal{S}}} \right)^p \left(\frac{y - y_{\mathcal{S}}}{h_{\mathcal{S}}} \right)^q, \quad (7)$$

where $(x_{\mathcal{S}}, y_{\mathcal{S}})$ is any point in the stencil $\mathcal{S}$. We choose the stencil “center” point: the intersection point of two diagonal lines joining top left, bottom right and top right, bottom left corners. Assuming that each cell $E_{i,j} \in \mathcal{S} \subset \mathcal{T}_h$ is shape regular as in Definition ??, we define the stencil scale parameter $h_{\mathcal{S}} = \sum_{i \leq s, j \leq t} E_{i,j} / (s \times t)$ as the average diameter of all cells in the stencil. Introducing the normalized coordinates

$$\xi = \frac{x - x_{\mathcal{S}}}{h_{\mathcal{S}}}, \quad \text{and} \quad \eta = \frac{y - y_{\mathcal{S}}}{h_{\mathcal{S}}}. \quad (8)$$

We can rewrite equation (7) in normalized coordinates as

$$\hat{Q}_{s,t}^{\mathcal{S}}(\xi, \eta) = \sum_{p < s, q < t} c_{p,q} \xi^p \eta^q. \quad (9)$$

The tensor product stencil polynomial satisfies the condition that the cell-averaged value (3) is preserved through integral over each cell as

$$\frac{1}{|E_{i,j}|} \iint_{E_{i,j}} Q_{s,t}^s(x, y) dx dy = \bar{u}_{i,j}, \quad \forall E_{i,j} \in \mathcal{S}, \quad (10)$$

where $\bar{u}_{i,j}$ represents the cell-averaged scalar variable over cell $E_{i,j}$.

One way to construct a stencil polynomial $Q_{s,t}^s$ is using basis polynomials. Let $Q_{s,t}^s = \sum_{i,j} \bar{u}_{i,j} \phi_{s,t}^{i,j}$, where the basis polynomials $\phi_{s,t}^{i,j}$ satisfy

$$\frac{1}{|E_{i',j'}|} \iint_{E_{i',j'}} \phi_{s,t}^{i,j}(x, y) dx dy = \begin{cases} 1 & (i', j') = (i, j) \\ 0 & (i', j') \neq (i, j) \end{cases}, \quad \forall E_{i',j'} \in \mathcal{S}, \quad (11)$$

which would always lead to a $(s \times t) \times (s \times t)$ non-singular linear system. We can write the basis polynomials in terms of normalized coordinates ξ and η as well, and rewrite it as

$$\hat{Q}_{s,t}^s(\xi, \eta) = \sum_{i,j} \bar{u}_{i,j} \hat{\phi}_{s,t}^{i,j}(\xi, \eta). \quad (12)$$

In most two-dimensional cases, we implement a square stencil with $s = t = r$. The approxmatibility of a tensor product polynomial is related to the associated complete polynomial, as discussed by Dupont and Scott (1980). Given a bounded interpolation projection operator $\mathcal{Q}$, for all $\mathbf{x} \in E \in \mathcal{S}$, we have

$$|D^\alpha u(\mathbf{x}) - D^\alpha(\mathcal{Q}u(\mathbf{x}))| \leq \begin{cases} Ch^{r-|\alpha|} & \text{if } u \text{ is smooth on the stencil,} \\ C & \text{otherwise,} \end{cases} \quad (13)$$

where $\alpha = (\alpha_1, \alpha_2)$ is a two-dimensional multi-index, and $|\alpha| = \alpha_1 + \alpha_2 \leq r$. The uneven stencils ($s \neq t$) are used in the following two situations:

- When some a prior knowledge of the solution is acquired before simulation, e.g., the transport velocity is known to be fixed in a single direction.
- When derivatives are approximated, e.g., diffusive flux is computed across the element edge, where a stretched stencil is necessary to compensate for the loss of accuracy associated with derivative approximation.

For simplicity, we restrict the following discussions to the square case $s = t = r$.

0.2.2 Stencil polynomial smoothness indicators

A smoothness indicator σ is defined to quantify the smoothness of the tensor product polynomial $Q_{r,r}^{\mathcal{S}}$ on the stencil $\mathcal{S}$. The classical approach by Jiang and Shu (1996) defines σ is in analogue to a Sobolev seminorm as:

$$\sigma_{JS} = \sum_{1 \leq |\alpha| \leq r} \int \int_{E_0} \frac{h_{\mathcal{S}}^{2|\alpha|}}{|E_0|} (\mathcal{D}^{\alpha} Q(x, y))^2 dx dy, \quad (14)$$

The Jiang-Shu smoothness indicator can be computed utilizing predefined basis polynomials (11):

$$\begin{aligned} \sigma_{JS} &= \sum_{1 \leq |\alpha| \leq r} \frac{h_{\mathcal{S}}^{2|\alpha|}}{|E_0|} \iint_{E_0} (\mathcal{D}^{\alpha} Q(x, y))^2 dx dy \\ &= \frac{h_{\mathcal{S}}^2}{|E_0|} \sum_{1 \leq |\alpha| \leq r} \iint_{\hat{E}_0} (\mathcal{D}^{\alpha} \hat{Q}(\xi, \eta))^2 d\xi d\eta \\ &= \frac{h_{\mathcal{S}}^2}{|E_0|} \sum_{1 \leq |\alpha| \leq r} \iint_{\hat{E}_0} \left(\sum_{ij} \bar{u}_{ij} \mathcal{D}^{\alpha} \hat{\phi}_{i,j}(\xi, \eta) \right)^2 d\xi d\eta \\ &= \frac{h_{\mathcal{S}}^2}{|E_0|} \sum_{ij} \sum_{i'j'} \bar{u}_{ij} \bar{u}_{i'j'} \sum_{1 \leq |\alpha| \leq r} \iint_{\hat{E}_{ij}} \mathcal{D}^{\alpha} \hat{\phi}_{ij}(\xi, \eta) \mathcal{D}^{\alpha} \hat{\phi}_{i'j'}(\xi, \eta) d\xi d\eta \\ &= \frac{h_{\mathcal{S}}^2}{|E_0|} \sum_{ij} \sum_{i'j'} \bar{u}_{ij} \bar{u}_{i'j'} \sigma_{ij}^{i'j'}, \end{aligned} \quad (15)$$

where the coefficient tensor $\sigma_{ij}^{i'j'}$ can be precomputed offline after mesh is established.

Alternatively, we can compute the Jiang-Shu smoothness indicator directly using the polynomial coefficients formulation (9) as

$$\begin{aligned} \sigma_{JS} &= \frac{h_{\mathcal{S}}^2}{|E_0|} \sum_{1 \leq |\alpha| \leq r} \iint_{\hat{E}_0} (\mathcal{D}^{\alpha} \hat{Q}(\xi, \eta))^2 d\xi d\eta \\ &= \sum_{|\gamma| \leq r} \sum_{|\beta| \leq r} c_{\gamma} c_{\beta} \left(\sum_{1 \leq |\alpha| \leq r} \frac{h_{\mathcal{S}}^2}{|E_0|} \iint_{E_0} \mathcal{D}^{\alpha} \xi^{\gamma} \mathcal{D}^{\alpha} \eta^{\beta} d\xi d\eta \right) \\ &= \sum_{|\gamma| \leq r} \sum_{|\beta| \leq r} c_{\gamma} c_{\beta} \zeta_{\gamma, \beta}, \end{aligned} \quad (16)$$

which paves the way for further simplification.

The smoothness indicator exhibits the following convergence property: there exists a constant $C > 0$ such that when $h \rightarrow 0^+$,

$$\sigma = \begin{cases} Ch^2 + \mathcal{O}(h^3) & \text{Smooth,} \\ \Theta(1) & \text{Not Smooth.} \end{cases} \quad (17)$$

0.2.3 Two alternative smoothness indicator

We propose two alternative ways of defining a smoothness indicator

- *Simple smoothness indicator* Huang et al. (2023)

$$\sigma_{\text{simp}} = \frac{1}{N_s - 1} \sum_k (\bar{u}_k - \bar{u}_0)^2, \quad (18)$$

where $\bar{u}_0$ is the cell averaged solution in target cell. This simple smoothness indicator is designed to reduce the computational cost and to deal with distorted stencil on unstructured meshes. However, this simple smoothness indicator is not as robust as the classic Jiang-Shu smoothness indicator.

- *Polynomial smoothness indicator* Arbogast et al. (2025)

$$\sigma_P = \sum_{\alpha} \zeta_{\alpha, \alpha} c_{\alpha}^2 \quad (19)$$

which is obtained by dropping all the cross terms appeared in the formulation (16). This alternative serves as a good approximation to the classic Jiang-Shu smoothness indicator while the computing efficiency is greatly improved.

0.2.4 ML-WENO weighting

Given a target cell $E_{i,j}$, we can create a nonempty set of stencils $\{\mathcal{S}_l \ni E_{i,j}, l = 0, 1, \dots, L, L \geq 0\}$. The reconstruction of u on the target cell $E_{i,j}$ is a convex combination of the stencil polynomials defined in equation (12)

$$\mathcal{R}(\mathbf{x}) = \sum_l \tilde{\omega}_l Q^{\mathcal{S}_l}(\mathbf{x}), \quad \mathbf{x} \in E_{i,j}, \quad (20)$$

where $\sum_l \tilde{\omega}_l = 1$ is the normalized nonlinear weights associated with stencils accordingly. We begin by arbitrarily choosing a set of linear weights $\{\omega_l\}$ and scale them with stencil smoothness indicator as

$$\hat{\omega}_l = \frac{\omega_l}{(\sigma_l + \epsilon h_0^2)^{ar_l + n_l}}, \quad (21)$$

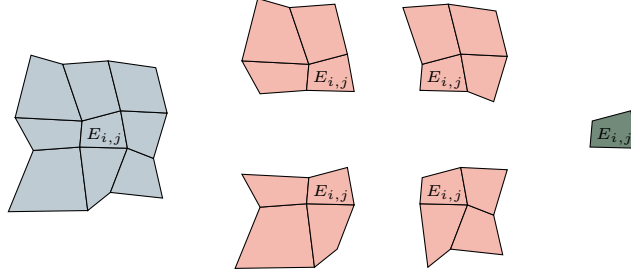


Figure 1: An example of (3,2,1) multi-level stencils combination on logically rectangular quadrilateral mesh.

where $r_l = \max\{s, t\} = r$ in the cases of using square stencils, and the constants $\epsilon = 1e - 4$ and $a > 0.5$ are chosen accordingly. The biasing factor $n_l = n(r_l)$ is used to biasing away further from the constant level, whose smoothness indicator is always zero, when the solution is smooth. We pick

$$n(r_l) = n_l = \begin{cases} 1, & \text{if } r_l = 1, \\ 3, & \text{if } r_l = 2, \\ 4, & \text{if } r_l \geq 3. \end{cases} \quad (22)$$

We then renormalise scaled weights as

$$\tilde{\omega}_l = \frac{\hat{\omega}_l}{\sum_{l'} \hat{\omega}_{l'}}. \quad (23)$$

0.2.5 The ML-WENO Reconstruction Error

In this section, we analyze the accuracy of our multi-level weighting scheme. We regroup the set of indices $l \in \mathcal{L} = \{0, 1, 2, \dots, L\}$ according to the order of the stencil polynomials. Specifically, we denote the set of indices of stencils have s cells in x direction and t cells in y direction as $\mathcal{L}_{s,t} \subset \mathcal{L}$. We assume the mesh resolution is sufficiently fine such that at least one smooth stencil exists in the presence of shocks. Accordingly, we define two sets holding indices of stencils with or without jump as

$$\begin{aligned} \text{Smooth} &= \{l : u \text{ is smooth on } \mathcal{S}_l, \} \\ \text{Jump} &= \{l : u \text{ has a jump on } \mathcal{S}_l\}. \end{aligned} \quad (24)$$

We assume that $\text{Smooth} \cup \text{Jump} = \mathcal{L}$ and $\text{Smooth} \cap \text{Jump} = \emptyset$. We represent the set of indices of stencils of size $(s \times t)$ that is smooth as $\mathcal{L}_{s,t} \cap \text{Smooth}$, and the set of stencils of size $(s \times t)$ that has a jump as $\mathcal{L}_{s,t} \cap \text{Jump}$. We show that the reconstruction is accurate with respect to the target cell E_0 to order

$$r_{\max} = \max\{r : \text{Smooth} \cap \mathcal{L}_{r,r} \neq \emptyset\}. \quad (25)$$

In Figure 2, we illustrate a (3,2,1) combination of stencils with two shocks present. Here, $\text{Smooth} = \{0, 1\}$, which indicates that stencil $\mathcal{S}_0$ and $\mathcal{S}_1$ do not have jump present. In this case, we expect the accuracy of reconstruction is $r_{\max} = \text{Smooth} \cap \mathcal{L}_{2,2} = 2$.

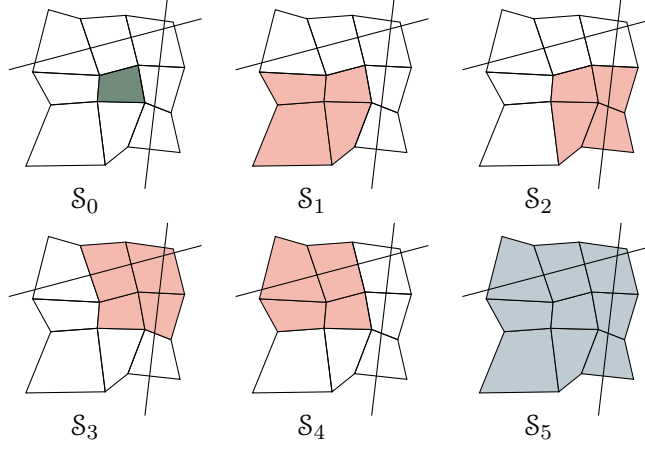


Figure 2: An illustration of a (3,2,1) multi-level stencils combination on logically rectangular quadrilateral mesh with two shock presents.

We follow the estimation of reconstruction error show in Arbogast et al. (2024). We start by estimating the error of our nonlinear scaling (21) associated with some stencils $\mathcal{S}_l$ for some $m > 0$.

$$\hat{\omega}_l = \frac{\omega_l}{(\sigma_l + \epsilon h_0^2)^m}. \quad (26)$$

We estimate the reconstruction error for two situtaions.

- $l \in \text{Smooth}$, the smoothness indicator $\sigma_l = Ch^2 + \mathcal{O}(h^3)$;
- $l \in \text{Jump}$, the smoothness indicator $\sigma_l = \Theta(1)$.

Use the asymptotic estimation result from Olver (1997). Let $n \geq m$, $a > 0$, and $b > 0$

$$\begin{aligned} \frac{a}{bh^m + \mathcal{O}(h^n)} &= \frac{ah^{-m}}{b}(1 + \mathcal{O}(h^{n-m})), \\ \frac{a}{bh^{-n} + \mathcal{O}(h^{-m})} &= \frac{ah^n}{b}(1 + \mathcal{O}(h^{n-m})). \end{aligned} \quad (27)$$

For the smooth stencil $\mathcal{S}_l \in \text{Smooth}$

$$\begin{aligned}\hat{\omega}_l &= \frac{\omega_l}{(\sigma_l + \epsilon h_0^2)^m} = \frac{\omega_l}{((C + \epsilon)h_0^2 + \mathcal{O}(h^3))^m} \\ &= \omega_l \left(\frac{h_0^{-2}}{C + \epsilon} (1 + \mathcal{O}(h)) \right)^m \\ &= \frac{\omega_l}{(D + \epsilon)^m} h_0^{-2m} (1 + \mathcal{O}(h)).\end{aligned}\tag{28}$$

In conclusion, we estimate the nonlinear scaling as

$$\hat{\omega}_l = \begin{cases} \frac{\omega_l}{(C + \epsilon)^m} h_0^{-2m} (1 + \mathcal{O}(h)), & \text{if } l \in \text{Smooth}, \\ \Theta(1), & \text{if } l \in \text{Jump}. \end{cases}\tag{29}$$

Then we estimate the error for the normalized nonlinear weighting (23). We expand the sum of nonlinear scaling as

$$\sum_l \hat{\omega}_l = \sum_r \sum_{l \in \text{Smooth} \cap \mathcal{L}_{r,r}} \hat{\omega}_l + \sum_{l \in \text{Jump}} \hat{\omega}_l.\tag{30}$$

We estimate the smooth part of the nonlinear weighting as

$$\sum_r \sum_{l \in \text{Smooth} \cap \mathcal{L}_{r,r}} \hat{\omega}_l = \sum_r \sum_{l \in \text{Smooth} \cap \mathcal{L}_{r,r}} \frac{\omega_l}{(C + \epsilon)^m} h_0^{-2(ar + \eta)} (1 + \mathcal{O}(h)).\tag{31}$$

Assume $h < 1$, we keep the largest $ar + \eta(r)$ number as our estimator

$$\begin{aligned}\sum_r \sum_{l \in \text{Smooth} \cap \mathcal{L}_{r,r}} \hat{\omega}_l &= \sum_{l \in \text{Smooth} \cap \mathcal{L}_{r_{\max}, r_{\max}}} \frac{\omega_l}{(C + \epsilon)^{ar_{\max} + \eta(r_{\max})}} h_0^{-2(ar_{\max} + \eta(r_{\max}))} (1 + \mathcal{O}(h)) \\ &\quad + \Theta(h^{-2(ar + \eta(r))}) \\ &= \sum_{l \in \text{Smooth} \cap \mathcal{L}_{r_{\max}, r_{\max}}} \frac{\omega_l}{(C + \epsilon)^{ar_{\max} + \eta(r_{\max})}} h_0^{-2(ar_{\max} + \eta(r_{\max}))} \\ &\quad + \mathcal{O}(h^{1-2(ar_{\max} + \eta(r_{\max}))}).\end{aligned}\tag{32}$$

later, we denote the set $\text{Smooth} \cap \mathcal{L}_r$ as Smooth_r for simplicity. The overall accuracy estimation of the nonlinear weighting (23) is presented as

- $l \in \text{Smooth}$

$$\tilde{\omega}_l = \frac{\hat{\omega}_l}{\sum_{l'} \hat{\omega}_{l'}} = \frac{\omega_l}{\sum_{l' \in \text{Smooth}_r} \omega_{l'}} h^{2[a(r_{\max} - r) + \eta(r_{\max}) - \eta(r)]} (1 + \mathcal{O}(h));\tag{33}$$

- $l \in \text{Jump}$

$$\tilde{\omega}_l = \frac{\hat{\omega}_l}{\sum_{l'} \hat{\omega}_{l'}} = \Theta(h^{2[a(r_{\max}) + \eta(r_{\max})]}). \quad (34)$$

Now we can estimate the reconstruction error as

$$\begin{aligned} |u(\mathbf{x}) - \mathcal{R}(\mathbf{x})| &= \left| \sum_l \tilde{\omega}_l (u(\mathbf{x}) - Q_l(\mathbf{x})) \right| \\ &\leq \sum_{l \in \text{Smooth}} \tilde{\omega}_l |u(\mathbf{x}) - Q_l(\mathbf{x})| + \sum_{l \in \text{Jump}} \tilde{\omega}_l |u(\mathbf{x}) - Q_l(\mathbf{x})| \\ &\leq \sum_r \sum_{\substack{l \in \\ \text{Smooth} \cap \mathcal{L}_{r,r}}} \Theta(h^{2[a(r_{\max}-r) + (\eta(r_{\max}) - \eta(r))]}) \mathcal{O}(h^r) \\ &\quad + \sum_{l \in \text{Jump}} \Theta(h^{2(ar_{\max} + \eta(r_{\max}))}) \mathcal{O}(1) \\ &= \mathcal{O}(h^{r_{\max}}) \end{aligned} \quad (35)$$

Combining everything above and (20), we arrive at the formal error estimate of the reconstruction.

Lemma 0.1. *Let $\mathcal{R}(\mathbf{x})$ be the reconstruction. Assuming a quasi-uniform mesh, $h_0 = \Theta(h)$, there exists a constant $C > 0$ such that for any $\mathbf{x} \in E_0$,*

$$|u(\mathbf{x}) - \mathcal{R}(\mathbf{x})| \leq Ch^{r_{\max}}, \quad (36)$$

where $r_{\max}$ is defined by (25).

0.2.6 Handling Boundary Conditions

In our simulation, we couple FEM and FVM solver on a single mesh. Therefore, accurate and compatible boundary condition should be assigned accordingly. However, it is usually difficult to manage boundary condition for higher order finite volume and finite difference method, which require wide stencils. Some deal with the difficulty use ghost cells or points outside the boundary, and maintain the same stencil as if they were in the interior of the domain. However, the extrapolation or estimation of the manufactured values of ghost cells or ghost points are not straightforward. A notable method, the inverse Lax-Wendroff procedure, substitute the normal derivative of the solution with tangential and time derivatives using the PDE relation in an iterative way to impose more precise values for the ghost cells or points. The key to the above strategy is to have accurate and stable reconstruction near the boundary. ML-WENO gives the flexibility to add stencils of the desired size that span back into the domain.

In order to assign proper boundary condition to a FVM discretization, we should determine whether the flow is inflow or outflow first. If $\mathbf{f}(u_B) \cdot \boldsymbol{\nu} > 0$, we recognize that it is a outflow boundary, therefore, no fixed boundary condition should be assigned. We use "free outflow" boundary that the numerical flux is simply $\mathbf{f}(u_B) \cdot \boldsymbol{\nu}$.

The inflow boundary can be understood in various ways. We can understand it as prescribing the inflow flux $\mathbf{f}_B$ or fixing the boundary value u_B as an equivalent to Dirichlet boundary in the context of Finite Element. In the special case when a zero flux is prescribed, we would call it noflow boundary. In the inflow case, there may be a discontinuous inflow shock wave. We require a swift response inside the domain to align with the specified Dirichlet value. We therefore add an extra constant stencil polynomial to the inflow boundary to accommodate the potential discontinuity.

There are some other special cases that require special treatment on the boundary, e.g., when symmetry should be enforced.

0.2.7 Numerical Test

For completeness, we test our ML-WENO algorithm with simple Riemann shock and rarefaction. Let $\mathbf{f}(u) = (u^2/2, u^2/2)$ be the Burgers flux function. In this example, we take the initial condition $u_0(x, y) = 1$, and the computational domain $\Omega = [0, 3] \times [0, 1]$. The true solution is

$$u(x, y, t) = \begin{cases} 0, & x < 0.5, x \geq 0.5t + 1.5, \\ (x - 0.5)/t, & 0.5 \leq x < t + 0.5, \\ 1, & t + 0.5 \leq x < 0.5t + 1.5. \end{cases} \quad (37)$$

We forward marching to the time = 2.0, when trailing rarefaction reaches the leading shock. We use SSP2RK time stepping scheme.

Works Cited

Todd Arbogast, Chieh-Sen Huang, and Chenyu Tian. A finite volume multilevel weno scheme for multidimensional scalar conservation laws. *Comput. Methods Appl. Mech. Engrg.*, 421, 2024.

Todd Arbogast, Chieh-Sen Huang, Chenyu Tian, and Gabirel M.Gray. An efficient and accurate approximation to the classic polynomial smoothness indicator. *In Preparation.*, 2025.

J.H. Bramble and S. R. Hilbert. Estimation of linear functionals on Sobolev spaces with application to Fourier transform and spline interpolation. *SIAM J. Numer. Anal.*, 7(1):112–124, 1970. doi: 10.1137/0707006.

Todd Dupont and Ridgway Scott. Polynomial approximation of functions in Sobolev spaces. *Mathematics of Computation*, 34(150):441–463, 1980.

Chieh-Sen Huang, Todd Arbogast, and Chenyu Tian. Multidimensional weno-ao reconstructions using a simplified smoothness indicator and applications to conservation laws. *J. Sci. Comput.*, 97, 2023.

Guang-Shan Jiang and Chi-Wang Shu. Efficient implementation of weighted eno schemes. *J. Comput. Phys.*, 126:202–228, 1996.

Frank Olver. *Asymptotics and special functions*. AK Peters/CRC Press, 1997.